

스마트 물류 로봇팔 2 — Jetson 런타임 정리

기준 경로:

```
/home/aidl/work/rack_dataset_v1/jetson_runtime_bundle
```

이 폴더는 시연 시 필요한 Jetson 실행 코드, TensorRT 엔진, 카메라 설정, ROI 기준 이미지와 OpenRB 펌웨어를 모아 둔 실행용 스냅샷이다. 원본 rack_dataset_v1 파일을 이후 수정해도 이 번들에는 자동 반영되지 않는다.

1. 시스템 전체 흐름

Logitech C270

-> Jetson TensorRT: 3x3 슬롯 EMPTY/OCCUPIED 판정

-> 웹 UI와 /status.json 제공 (포트 8080)

FPGA Pcam 5C

-> 도형 + 물체 중심좌표 8바이트 UART 전송

-> Jetson이 유효 패킷마다 즉시 ACK

Jetson 통합 컨트롤러

-> 도형에 대응하는 열 선택

-> 카메라가 확인한 가장 낮은 EMPTY 슬롯 선택

-> 물체 중심좌표 + 슬롯 번호를 OpenRB에 9바이트로 전송

OpenRB-150

-> ACK 전송

-> 물체 집기 및 지정 슬롯 적재

-> 완료 시 DONE, 실패 시 FAULT 전송

Jetson은 카메라 판정이 불확실하거나 열이 가득 찼을 때 로봇 명령을 보내지 않는 fail-closed 방식으로 동작한다.

2. 도형과 적재 위치

도형별 열은 고정되어 있다.

FPGA shape	이름	적재 열
0	NONE	명령 없음
1	CIRCLE	C1, 왼쪽

2	SQUARE	C2, 가운데
3	TRIANGLE	C3, 오른쪽

카메라에서 적재함을 정면으로 봤을 때 슬롯 번호는 다음과 같다.

```
상단 L3: 7 8 9
중단 L2: 4 5 6
하단 L1: 1 2 3
C1 C2 C3
```

각 열에서 L1 -> L2 -> L3 순서로 가장 낮은 EMPTY 슬롯을 선택한다. 예를 들어 CIRCLE이고 C1_L1이 차 있고 C1_L2가 비어 있으면 슬롯 4를 보낸다.

3. 실제 런타임 파일

메인 실행 파일

파일	역할
tools/deployment/live_rack_monitor.py	카메라 촬영, TensorRT 추론, 정렬 검사, 웹 UI와 상태 JSON 제공
tools/communication/robot_dispatch_controller.py	FPGA 수신, 슬롯 선택, OpenRB 명령 및 ACK/DONE 통합 제어
final_openrb_20260810/final_openrb_20260810.ino	OpenRB에 업로드할 로봇팔 펌웨어

통합 컨트롤러의 필수 모듈

파일	역할
zybo_target_receiver_8byte.py	FPGA 8바이트 패킷 검증·해석; 단독 UART 점검도 가능
zybo_slot_receiver.py	동일 도형 3회 확인과 NONE 3회 재활성화 이벤트 게이트
slot_selector.py	도형-열 매칭 및 가장 낮은 EMPTY 슬롯 선택
openrb_target_transport.py	OpenRB 9바이트 좌표+슬롯 명령, 재시도, ACK/DONE 대기
openrb_transport.py	공통 CRC8, 상태 코드 및 5바이트 응답 파서

카메라 모니터의 필수 모듈과 데이터

파일	역할
check_rack_alignment.py	기준 영상과 현재 적재함 위치 정렬 검사
infer_rack_frame_trt.py	9개 ROI 전처리와 TensorRT 판정 지원

rack_safety_gate.py	5프레임 시간 합의와 UNKNOWN 안전 처리
validate_tensorrt_parity.py	TensorRT 실행기와 공통 전처리 함수 제공
config/camera_c270_live.json	C270 해상도, FPS, 노출·밝기 등 카메라 설정
deployment/.../engine/*.plan	MobileNetV3-Small TensorRT FP16 엔진
deployment/.../source/deployment_metadata.json	ROI, 전처리, 모델 메타데이터
deployment/.../source/thresholds.json	EMPTY/OCCUPIED 확정 임계값
raw/s01_train/c270_calv1_s01_l00_a001.jpg	적재함 정렬 기준 이미지

__pycache__는 Python이 자동 생성하는 캐시이며 실행 원본이 아니다.

4. UART 패킷

FPGA -> Jetson: 8바이트

```
[0] AA   Header
[1] SHAPE 0=NONE, 1=CIRCLE, 2=SQUARE, 3=TRIANGLE
[2] X_H   중심 X 상위 바이트
[3] X_L   중심 X 하위 바이트
[4] Y_H   중심 Y 상위 바이트
[5] Y_L   중심 Y 하위 바이트
[6] 00   Reserved
[7] CHECKSUM bytes[1]~bytes[6] XOR
```

- UART: 115200 8-N-1
- 좌표: X/Y 모두 big-endian, X=0..1919, Y=0..1079
- Jetson 응답: 정상 06 ACK, 오류 15 NAK
- 정상 패킷은 도형 이벤트 생성 여부와 무관하게 매번 ACK한다.
- 같은 도형 3회 연속 수신 시 물체 이벤트 한 번을 만든다.
- 이벤트 후 모든 non-NONE을 무시하며, NONE 3회 연속 수신 후 재활성화한다.
- 따라서 FPGA가 첫 ACK를 받은 뒤 해당 물체 전송을 완전히 중단하면 이벤트가 만들어지지 않는다. FPGA는 같은 검출 결과를 최소 3개의 정상 패킷으로 계속 보고하고, 물체가 사라지면 NONE도 최소 3개 보고해야 한다.

Jetson -> OpenRB: 9바이트

```
AA 55 COMMAND_ID SLOT X_H X_L Y_H Y_L CRC8
```

- CRC-8/ATM: polynomial 0x07, initial 0x00

- CRC 범위: COMMAND_ID부터 Y_L까지 6바이트
- COMMAND_ID: 프로그램 시작 시 현재 시각의 하위 8비트로 시작하고, OpenRB 전송을 시도한 명령마다 1씩 증가하며 255 다음은 0이다.
- 재시도 시 같은 command ID와 같은 전체 프레임을 다시 보낸다.

OpenRB -> Jetson: 5바이트

AA 55 COMMAND_ID STATUS CRC8

STATUS	의미
06	ACK: 명령 수신·검증·작업 등록 완료
07	DONE: 로봇 동작 정상 완료
15	NAK: 프레임, CRC, 슬롯 또는 좌표 오류
E0	BUSY: 다른 작업 수행 중이거나 자동 모드 꺼짐
E1	FAULT: 로봇 동작 실패

Jetson은 ACK만으로 성공 처리하지 않고 반드시 DONE까지 확인한다. 기본 설정은 ACK 1초, DONE 60초, 최초 1회와 재시도 2회를 합쳐 최대 3번의 전송이다.

5. 카메라 판정과 안전 조건

각 ROI의 TensorRT 출력은 OCCUPIED 확률 p_occupied이다.

```
p <= 0.005 -> EMPTY
p >= 0.995 -> OCCUPIED
그 사이    -> UNKNOWN
```

5프레임 합의가 완료되고 다음 조건을 모두 만족해야 슬롯을 선택한다.

- 모니터 상태가 STABLE
- 적재함 pose가 aligned=true
- 9개 슬롯 상태가 모두 존재
- 어떤 슬롯도 UNKNOWN이 아님
- 해당 도형 옆에 EMPTY 슬롯이 있음

조건 하나라도 실패하면 OpenRB 명령을 전송하지 않는다. 카메라 밝기 설정은 학습 시 조건을 유지한다. 웹 미리보기의 0.82 밝기 보정은 표시 전용이며 TensorRT 입력 영상에는 적용되지 않는다.

현재 시연에서 사용한 정렬 완화 옵션은 다음과 같다.

```
--max-allowed-shift 24
```

```
--minimum-alignment-score 0.25
--alignment-interval 0.5
```

기준 위치 변화를 더 많이 허용하는 대신 잘못된 ROI를 정상 정렬로 받아들이는 가능성이 커지므로, 카메라와 적재함 각도를 표시하고 시연 전 빈 칸/점유 칸을 모두 확인해야 한다.

6. 하드웨어 연결

USB-UART 두 개가 필요하다. /dev/ttyUSB0과 /dev/ttyUSB1 번호는 연결 순서에 따라 바뀔 수 있으므로 매번 용도를 확인한다.

FPGA USB-TTL

```
FPGA TX -> USB-TTL RX
FPGA RX <- USB-TTL TX
FPGA GND -- USB-TTL GND
```

OpenRB USB-TTL

```
USB-TTL TX -> OpenRB D13 (Serial3 RX)
USB-TTL RX <- OpenRB D14 (Serial3 TX)
USB-TTL GND -- OpenRB GND
USB-TTL +5V 빨간 선은 연결하지 않음
```

OpenRB 스케치는 Serial3, 115200 8-N-1로 설정되어 있다. .ino 파일을 번들에 복사한 것만으로 보드에 적용되지는 않으며 Arduino IDE에서 실제 OpenRB-150에 업로드해야 한다.

7. 시연 실행 순서

먼저 번들 경로로 이동한다.

```
cd /home/aidl/work/rack_dataset_v1/jetson_runtime_bundle
```

장치를 확인한다.

```
ls -l /dev/video0
ls -l /dev/ttyUSB*
```

터미널 1: 카메라와 웹 UI

```
python3 tools/deployment/live_rack_monitor.py \
--max-allowed-shift 24 \
```

```
--minimum-alignment-score 0.25 \  
--alignment-interval 0.5
```

브라우저에서 아래 주소를 연다.

```
http://JETSON_IP:8080/
```

Jetson 터미널 자체에서는 다음으로 상태를 확인할 수 있다.

```
curl -fsS http://127.0.0.1:8080/status.json
```

터미널 2: 먼저 DRY-RUN

아래 예시는 FPGA가 /dev/ttyUSB1일 때이다.

```
python3 tools/communication/robot_dispatch_controller.py \  
--fpga-port /dev/ttyUSB1 \  
--dry-run
```

[FPGA EVENT], [CAMERA], [SELECT], [DRY-RUN]이 순서대로 나오는지 확인하고 Ctrl+C로 종료한다.

터미널 2: 최종 통합 실행

아래 예시는 FPGA가 /dev/ttyUSB1, OpenRB가 /dev/ttyUSB0일 때이다.

```
python3 tools/communication/robot_dispatch_controller.py \  
--fpga-port /dev/ttyUSB1 \  
--openrb-port /dev/ttyUSB0 \  
--openrb-done-timeout 60
```

포트가 반대이면 두 인수를 서로 바꿔야 한다. 다음 수신기를 통합 컨트롤러와 동시에 실행하면 같은 FPGA UART를 두 프로그램이 경쟁하므로 금지한다.

```
zybo_target_receiver_8byte.py  
zybo_slot_receiver.py  
xxd, minicom 등 동일 FPGA 포트를 여는 프로그램
```

FPGA Vitis와 Jetson 프로그램 중 어느 쪽을 먼저 실행해도 되지만, 시연에서는 Jetson 카메라 모니터 -> Jetson 통합 컨트롤러 -> FPGA 전송 순서가 로그를 확인하기 쉽다.

8. 정상 로그

완전한 한 사이클은 대략 다음과 같다.


```
[QUEUED] event=1 shape=CIRCLE center=(542,513)
[FPGA EVENT] #1 shape=CIRCLE centroid=(542,513)
[CAMERA] status=STABLE ... aligned=True ...
[SELECT] shape=CIRCLE column=C1 slot_name=C1_L1 slot=1
[OPENRB TARGET TX] attempt=1/3 command_id=... slot=1 center=(542,513) ...
[OPENRB RX] command_id=... status=ACK
[OPENRB RX] command_id=... status=DONE
[DISPATCH DONE] ...
[REARM] NONE 확인; 다음 물체를 받을 준비가 되었습니다.
```

통합 컨트롤러는 FPGA ACK를 빠르게 반환하기 위해 UART 수신과 OpenRB 로봇 작업을 별도 스레드로 처리한다. 따라서 [QUEUED]가 먼저 나오고 실제 슬롯 선택과 로봇 동작 로그가 뒤이어 나오는 것이 정상이다.

9. 주요 차단 로그

로그/사유	의미 및 확인 사항
FPGA_TARGET_OUT_OF_RANGE	FPGA 좌표가 1920x1080 범위 밖
CAMERA_STATUS_UNAVAILABLE	웹 모니터 미실행 또는 8080 상태 API 접근 실패
CAMERA_NOT_STABLE:UNKNOWN	5프레임 상태가 확정되지 않음
CAMERA_POSE_NOT_ALIGNED	적재함 위치가 정렬 허용 범위 밖
UNKNOWN_OCCUPANCY	하나 이상의 슬롯이 EMPTY/OCCUPIED로 확정되지 않음
COLUMN_FULL	해당 도형의 열에 빈 슬롯이 없음
OPENRB_UART_FAILED	OpenRB 포트 분리, 권한 또는 드라이버 문제
OPENRB_RESPONSE_TIMEOUT	ACK 또는 DONE 미수신
OPENRB_BUSY	OpenRB가 기존 동작 중이거나 자동 모드가 꺼짐
OPENRB_FAULT	로봇 동작 실패

COLUMN_FULL이면 로봇 명령을 보내지 않고 웹 UI 이벤트 로그에도 해당 열 또는 전체 적재함이 가득 찼다는 메시지를 보낸다.

10. OpenRB 현재 설정 주의

현재 펌웨어에는 다음 설정이 들어 있다.

```
constexpr bool USE_FIXED_PICK_POSE = true;
```

따라서 UART X/Y는 정상 수신·검증되지만 실제 집기에는 고정 관절 자세를 쓰며, 슬롯 번호만 적재 동작에 사용된다. FPGA 중심좌표로 집기 자세를 계산하려면 이를 false로 바꾸고, 8점 보간 보정값과 로봇 안전 범위를 확인한 다음 OpenRB에 다시 업로드해야 한다.

통합 컨트롤러는 FPGA ACK가 늦어지지 않도록 새 물체 이벤트를 작업 큐에 넣고 OpenRB 명령은 한 개씩 순차 실행한다. 따라서 로봇 동작 중 새 물체가 도착하면 현재 구현은 그 이벤트를 즉시 BUSY로 폐기하지 않고 큐에 보관했다가 앞 작업의 DONE 이후 처리할 수 있다. 요구사항이 “로봇 동작 중 들어온 새 물체는 무조건 차단”이라면 컨트롤러의 큐 등록 조건을 추가로 수정해야 한다.

11. 시연 전 최종 체크

- C270가 /dev/video0으로 인식되는지 확인
- FPGA/OpenRB USB-UART 포트를 서로 구분
- OpenRB에 최신 .ino가 업로드됐는지 확인
- 카메라와 적재함 각도 및 거리 고정
- 웹 UI가 STABLE, aligned=YES, consensus 5/5인지 확인
- 실제 배치와 9개 슬롯 표시가 모두 일치하는지 확인
- FPGA 도형별 열 C1/C2/C3가 맞는지 DRY-RUN으로 확인
- OpenRB ACK와 DONE이 모두 수신되는지 확인
- 열이 가득 찬 경우 로봇 명령이 차단되는지 확인
- 종료는 각 터미널에서 Ctrl+C

이 번들은 현재 개발·시연 구성의 실행 스냅샷이며 산업 안전 인증이나 사람과 함께 운용하는 로봇 안전 시스템을 대신하지 않는다.